

Namal University FYP Portal - Academic Excellence

Project Overview

The Namal University Final Year Project (FYP) Portal is a robust, web-based management system meticulously designed to centralize and streamline the entire lifecycle of Final Year Projects. It serves as a comprehensive platform for students, faculty, project coordinators, and external evaluators to collaborate efficiently, manage project milestones, submit deliverables, and assess progress. The portal aims to enhance transparency, accountability, and overall academic excellence in the FYP process at Namal University.

This system is built with a focus on modularity and scalability, providing distinct interfaces and functionalities tailored to the specific roles involved in FYP management. From initial project proposals to final defense evaluations, the portal offers a structured environment to track every stage of a project.

Key Features by Role

The portal provides a role-based access control system, ensuring that each user interacts with functionalities relevant to their responsibilities:

Role	Description	Core Capabilities
Students	Undergraduate students undertaking their Final Year Project.	Access to project details, group management, milestone tracking, submission of reports/code/presentations, viewing evaluation feedback, and personal profile management.
Faculty	Academic staff acting as supervisors or co-supervisors for student projects.	Oversight of assigned projects, tracking student progress, reviewing and providing feedback on submissions, managing project supervision assignments, and accessing faculty-specific dashboards.
Coordinators	Administrative staff responsible for the overall FYP program.	Comprehensive management of batches, defining and scheduling project milestones, assigning students to groups, allocating faculty supervisors, and system-wide reporting.
Evaluators	Internal or external experts responsible for assessing project quality.	Access to assigned project submissions, application of marking rubrics, recording evaluation scores, and

		providing detailed feedback to students.
Super Admin	System administrators with full control over the portal's configuration and user base.	User account creation and management, role assignment, system-wide settings configuration, monitoring audit logs, and ensuring system integrity.

Technical Stack

The Namal FYP Portal is developed using a modern and efficient technology stack:

- **Frontend:** HTML5, CSS3, JavaScript (Vanilla) for dynamic and interactive user interfaces. The design leverages Tailwind CSS for utility-first styling and Material Symbols for iconography.
- **Backend:** Node.js with Express.js framework, providing a robust and scalable API layer for handling business logic and data interactions.
- **Database:** MySQL, a powerful open-source relational database management system, used for storing all project-related data, user information, and system configurations.
- **Authentication:** JSON Web Tokens (JWT) for secure, stateless authentication, coupled with Bcryptjs for strong password hashing and security.
- **File Handling:** Multer, a Node.js middleware for handling `multipart/form-data`, primarily used for uploading project submissions (reports, code, presentations).
- **Development Tools:** npm for package management, `dotenv` for environment variable management.

Prerequisites

To set up and run the Namal FYP Portal on your local machine, ensure you have the following software installed:

1. **Node.js:** Version 14.x or higher. You can download it from nodejs.org.
 - Verify installation: `node -v`
2. **npm:** Node Package Manager, which comes bundled with Node.js. Version 6.x or higher.
 - Verify installation: `npm -v`
3. **MySQL Server:** Version 8.0 or higher. You can download it from [mysql.com](https://www.mysql.com/downloads/).

- Ensure the MySQL server is running and you have access to a user with sufficient privileges (e.g., `root` user).

Installation & Setup Guide

Follow these detailed steps to get the Namal FYP Portal up and running on your local development environment.

1. Clone the Repository

First, clone the project repository to your local machine:

```
git clone <repository_url>
```

```
cd namal-fyp-portal
```

2. Database Configuration

This project uses a MySQL database named `fyb\_db`. You need to create this database and import the provided schema.

2. **Start MySQL Server:** Ensure your MySQL server is running.

2. **Access MySQL:** Open your MySQL client (e.g., MySQL Workbench, command-line client, or phpMyAdmin).

3. **Create Database:** Execute the following SQL command to create the database:

```
CREATE DATABASE fyb_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

```
USE fyb_db;
```

4. **Import Schema:** The complete database schema, including tables, views, stored procedures, functions, and triggers, is located in `database/schema\_with\_procedures\_triggers.sql`.

- Navigate to the project directory.
- Execute the SQL script using your MySQL client. For example, from the command line:

```
mysql -u your_mysql_user -p fyb_db < database/schema_with_procedures_triggers.sql
```

(Replace `your\_mysql\_user` with your MySQL username, and you will be prompted for the password.)

3. Environment Setup

Environment variables are used to manage sensitive information and configuration settings.

Create a file named `.env` in the root directory of your project. This file will store your database credentials and JWT secret.

```
# Server Port
```

```
PORT=3000
```

Database Configuration

DB_HOST=localhost

DB_USER=root

DB_PASS=your_mysql_password

DB_NAME=fyb_db

JSON Web Token Secret (for authentication)

JWT_SECRET=your_very_secret_key_here

Important:

Replace `your\_mysql\_password` with the actual password for your MySQL `root` user (or the user you configured).

****Replace `your\_very\_secret\_key\_here` with a strong, unique secret key. This is crucial for the security of your JWTs.***

**** If you choose not to use a `.env` file, you ****must**** manually update the database connection details in `src/config/db.js` and the `JWT\_SECRET` in `src/middleware/auth.js`.***

4. Install Node.js Dependencies

Navigate to the project's root directory in your terminal and install all required Node.js packages:

```
npm install
```

This command reads the `package.json` file and installs all listed `dependencies` (e.g., `express`, `mysql2`, `bcryptjs`, `jsonwebtoken`, `multer`, `dotenv`).

5. Initialize Database Objects (Post-Schema)

After importing the schema, run the `setup\_db.js` script. This script ensures that all necessary stored procedures, functions, triggers, and an `audit\_log` table are correctly created or updated in your `fyb\_db` database. It also adds any missing columns to the `user\_account` table and ensures a `superadmin` account exists.

```
node setup_db.js
```

This script is crucial for the full functionality of the backend, especially for features like user account management and system auditing.

6. Seed Demo Data (Optional but Recommended)

To populate your database with sample data for testing and development, run the ``seed_dummy_files.js`` script. This script inserts dummy entries into various tables (departments, batches, persons, students, faculty, projects, groups, milestones, submissions) and creates dummy physical files in the ``uploads/submissions/`` directory.

```
node scripts/seed_dummy_files.js
```

This will give you a functional dataset to explore the portal's features without manually entering data.

Running the Application

Once all the setup steps are completed, you can start the Node.js server:

```
node server.js
```

Upon successful startup, you will see messages in your console indicating that the server is running and the database connection is established. The application will typically be accessible at:

`http://localhost:3000`

Open this URL in your web browser to access the Namal FYP Portal.

Demo Credentials

If you ran the ``seed_dummy_files.js`` script, you can log in with the following credentials:

Role	Username	Password
Super Admin	superadmin	superadmin123
Faculty	faculty	faculty123
Coordinator	coordinator	coordinator123
Evaluator	evaluator	evaluator123

API Endpoints

The backend exposes various API endpoints to support the frontend functionalities. All API endpoints are prefixed with ``/api/``.

Endpoint Prefix	Description	Key Routes (Examples)
<code>/api/auth</code>	User authentication and authorization.	<code>`POST /api/auth/login`</code> (User login), <code>`GET /api/auth/me`</code> (Get current user info)
<code>/api/students</code>	Student-specific operations and data retrieval.	<code>`GET /api/students/:id/projects`</code> (Get student projects), <code>`POST /api/students/:id/submissions`</code> (Upload submission)
<code>/api/faculty</code>	Faculty-specific operations, project supervision.	<code>`GET /api/faculty/:id/supervisions`</code>

		(Get supervised projects), `PUT /api/faculty/:id/feedback` (Provide feedback)
/api/submissions	Management of project submissions.	`GET /api/submissions/:id` (Get submission details), `POST /api/submissions/upload` (Handle file uploads)
/api/evaluations	Evaluation-related operations.	`GET /api/evaluations/:id` (Get evaluation details), `POST /api/evaluations/:id/mark` (Submit marks)
/api/reports	Generation and retrieval of various reports.	`GET /api/reports/progress` (Project progress report), `GET /api/reports/milestones` (Milestone status report)
/api/coordinator	Coordinator-specific administrative tasks.	`POST /api/coordinator/batches` (Create new batch), `PUT /api/coordinator/milestones/:id` (Update milestone)
/api/superadmin	Super Admin system management.	`GET /api/superadmin/users` (Manage user accounts), `GET /api/superadmin/audit` (View audit logs)

Project Structure

The project is organized into a clear and logical directory structure to facilitate development and maintenance:

namal-fyp-portal/	
├── assets/	# Static assets (CSS, JS for frontend, images)
├── coordinator/	# HTML/JS/CSS files for the Coordinator dashboard and views
├── database/	# SQL schema, procedures, triggers, and seed data scripts
├── schema_with_procedures_triggers.sql	# Full database schema
├── evaluator/	# HTML/JS/CSS files for the Evaluator interface
├── faculty/	# HTML/JS/CSS files for the Faculty dashboard
├── node_modules/	# Installed Node.js packages (managed by npm)
├── scripts/	# Utility scripts (e.g., database seeding)
├── seed_dummy_files.js	# Script to populate database with sample data

— src/	# Backend source code
— config/	# Database connection configuration (db.js)
— controllers/	# Business logic for API endpoints (e.g., authController.js)
— middleware/	# Express middleware (e.g., auth.js for JWT validation)
— routes/	# API route definitions (e.g., authRoutes.js)
— student/	# HTML/JS/CSS files for the Student portal
— superadmin/	# HTML/JS/CSS files for the Super Admin panel
— uploads/	# Directory for storing user-uploaded files (submissions)
— submissions/	# Sub-directories for different submission types (code, reports, etc.)
— .env.example	# Example environment variables file
— package.json	# Project metadata and Node.js dependencies
— package-lock.json	# Records the exact dependency tree
— README.md	# This comprehensive setup guide
— server.js	# Main backend application entry point
— setup_db.js	# Database initialization and object creation script

Troubleshooting

- **Error: Can't connect to MySQL server:** Ensure your MySQL server is running and your DB_HOST, DB_USER, DB_PASS, and DB_NAME in the .env file (or src/config/db.js) are correct.
- **Error: JWT_SECRET is not defined:** Make sure you have a JWT_SECRET variable defined in your .env file or directly in src/middleware/auth.js.
- **npm install fails:** Check your Node.js and npm versions. Try clearing npm cache (npm cache clean --force) and reinstalling.
- **Missing tables/procedures:** Ensure you have successfully run database/schema_with_procedures_triggers.sql and node setup_db.js.
- **Frontend pages not loading:** Verify that server.js is running and serving static files correctly. Check the browser console for any errors.